

UNCERTAINTY-AWARE CEP FOR SOTS

HANDLING UNCERTAINTY IN COMPLEX EVENT
PROCESSING FOR SYSTEMS OF TWINNED SYSTEMS (SOTS)

BY
OLUWAFEYISAYO ADESANYA, B.Sc.

A THESIS
SUBMITTED TO THE COMPUTING & SOFTWARE
AND THE SCHOOL OF GRADUATE STUDIES
OF MCMASTER UNIVERSITY
IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
MASTER OF SCIENCE

© Copyright by Oluwafeyisayo Adesanya, April 2026

All Rights Reserved

Master of Science (2026)
(Computing & Software)

McMaster University
Hamilton, Ontario, Canada

TITLE: Handling Uncertainty in Complex Event Processing for
Systems of Twinned Systems (SoTS)

AUTHOR: Oluwafeyisayo Adesanya
B.Sc. (Hons) (Computer Science & Data Science),
University of Ottawa, Ottawa, Canada

SUPERVISOR: Dr. Istvan David

NUMBER OF PAGES: xi, 59

Abstract

Abstract here (no more than 300 words)

Your Dedication
Optional second line

Acknowledgements

Acknowledgements go here.

Contents

Abstract	iii
Acknowledgements	v
Notation, Definitions, and Abbreviations	x
Declaration of Academic Achievement	xi
1 Introduction	1
1.1 Big Picture	1
1.2 Shift to a New Class of Systems	2
1.3 Core Problem	4
1.4 Problem Statement	5
1.5 Contributions	7
1.6 Thesis Structure	8
2 Background	9
2.1 Digital Twins	9
2.2 Systems of Systems (SoS)	13
2.3 Complex Event Processing	13

3	Literature Review	14
3.1	Systems of Twinned Systems (SoTS)	14
3.2	Study Design	16
3.3	Survey of Systems of Twinned Systems	20
3.4	Reliability Mechanisms in SoTS	20
3.5	Complex Event Processing in DT and SoS Contexts	21
3.6	Research Gap and Contribution	21
4	Architecture	22
4.1	Overview	24
4.2	Framework Architecture	27
4.3	Messaging System	31
4.4	Event Lifecycle and System Flow	34
4.5	UML and Implementation	35
4.6	Scope and Limitations	37
4.7	Summary	38
5	Evaluation and Results	39
5.1	Evaluation Design	39
5.2	Results	50
6	Conclusion	52

List of Figures

3.1	Overview of the study design and selection workflow used for the SoTS systematic literature review.	17
4.1	Context diagram	24
4.2	Container-level Diagram	25
4.3	Coordinator layer performing scheduling and gap detection.	28
4.4	Flow of an event through the system	34
4.5	UML Class Diagram	36
5.1	Map of beaches	42
5.2	Per-beach attribute distributions (May–September 2015).	44
5.3	Hourly time-series overview for the three selected beaches (May–September 2015).	45
5.4	Illustration of a sample event composition of atomic and complex events	46

List of Tables

3.1	Search statistics	19
4.1	Topic hierarchy and routing semantics in the messaging system. . . .	32
5.1	Hierarchy of event patterns used in the evaluation.	47
5.2	Grid search parameter ranges for predictor tuning (Montrose Beach, 2016).	50
5.3	Comparison of reconstruction performance (MAE) per dataset and at- tribute (averaged across beaches) between Kalman and Particle filters under hybrid missingness.	51

Notation, Definitions, and Abbreviations

Notation

$A \leq B$ A is less than or equal to B

Definitions

Challenge With respect to video games, a challenge is a set of goals presented to the player that they are tasks with completing; challenges can test a variety of player skills, including accuracy, logical reasoning, and creative problem solving

Abbreviations

AI Artificial intelligence

Declaration of Academic Achievement

The student will declare his/her research contribution and, as appropriate, those of colleagues or other contributors to the contents of the thesis.

Chapter 1

Introduction

1.1 Big Picture

According to the International Council on Systems Engineering (INCOSE) Vision 2035 INCOSE (2023), the growing interconnectivity of cyber-physical systems and the drive toward autonomous operation are contributing to a sharp rise in system complexity. Governments are acknowledging this shift. In Canada, the Improving the Reliability and Resilience of Canada’s Digital Infrastructure report calls for reinforcing national data and communication systems to meet rising connectivity and information demands (Innovation, Science and Economic Development Canada and Canadian Forum for Digital Infrastructure Resilience, 2023).

This growing dependence on interconnected digital infrastructure parallels the global transition toward Industry 4.0 (Suleiman et al., 2022), where digital transformation has become a central driver of innovation across industries (Vial, 2021). Enterprises now integrate cyber-physical systems, Internet of Things (IoT) technologies, artificial intelligence (AI), and automation into their operations (Piardi et al.,

2024; Hu et al., 2024), making information and analytics essential to performance, sustainability, and decision support.

The integration of digital technologies into day-to-day operations is transforming both established and emerging sectors. In domains such as smart cities, transportation, and utilities, data is leveraged to coordinate systems and enable predictive management and maintenance (Tricomi et al., 2024; Chowdhury et al., 2024). Decision-making increasingly relies on real-time insights to support adaptive responses—for example, adjusting traffic signals, balancing power grids, or monitoring environmental conditions. Equally transformative are domains that have traditionally relied on physical or human-centered practices: precision agriculture employs sensor networks to optimize irrigation and crop yield (Chithrakumar et al., 2024), while healthcare applies large-scale analytics and AI-driven diagnostics for patient management and treatment design (Manikandan et al., 2024).

Across all domains, the availability, quality, and continuity of data directly influence system performance and decision-making. This growing dependency marks a fundamental shift—modern enterprises increasingly rely on algorithmic reasoning to interpret and act upon information. As the boundaries between physical and digital operations dissolve, ecosystems of interconnected, data-driven cyber-physical systems are emerging.

1.2 Shift to a New Class of Systems

A major enabler of the digital transformation shaping modern enterprises is the emergence of digital twins (DTs). DTs serve as high-fidelity virtual counterparts of physical entities, continuously updated through sensor data and computational models

to mirror their real-time state and behaviour (Kritzinger et al., 2018; Leng et al., 2021). They enable systems to be monitored, simulated, and optimized throughout their lifecycles, aligning with the goals of Industry 4.0 through data-informed decision-making and adaptive control (Pires et al., 2019). Through DTs, integration and semantic consistency between the physical and digital domains are achieved, enabling logical interpretation, representation, and interaction across both.

While DTs enable the development of complex and autonomous cyber-physical systems, their true potential emerges when they interconnect. In practice, multiple DTs often coexist within shared environments, exchanging information, coordinating actions, and influencing one another’s behaviour (Dietz and Pernul, 2020). Together, these interactions give rise to collaborative networks of twinned systems. This collective behaviour reflects the system of Systems (SoS) paradigm, in which autonomous entities cooperate to achieve objectives that exceed their individual purposes (Maier, 1998).

SoS are characterized by several recurring traits. They exhibit constituent independence, where each subsystem can operate and evolve autonomously while contributing to shared goals. They demonstrate evolutionary development, as their composition and configuration change dynamically when new systems join, existing ones depart, or interconnections are restructured. They also display emergent behaviour, arising from the interactions among heterogeneous constituents that produce outcomes not predictable from the behaviour of individual systems (Nielsen et al., 2015).

As networks of digital twins scale and become increasingly interconnected, they give rise to a new class of systems: Systems of Twinned Systems (SoTS) (Adesanya et al., 2025). SoTS unite the high-fidelity modelling and state awareness of DTs with

the organizational dynamics of SoS. This convergence supports enhanced situational awareness, coordinated decision-making, and adaptive operation across distributed, heterogeneous environments.

1.3 Core Problem

The integration of DTs within SoS architectures offers extraordinary potential for utilizing system-level intelligence. Through continuous sensing and interaction, SoTS enable distributed components to collaborate, negotiate, and coordinate complex behaviours across domains, supporting adaptive and goal-oriented decision-making at scale (Esterle et al., 2021). Yet the same characteristics that make SoTS powerful also compound challenges in maintaining data reliability and continuity.

Within SoTS, each constituent DT collects data from its own sensors to capture real-time information about the state and behaviour of its physical counterpart (Zhang et al., 2024). Together, these data flows form the informational backbone of each constituent and, when integrated at the SoS level, create an interconnected representation of the system that enables coordination and situational awareness. However, these data flows are inherently imperfect. Sensors and communication links are prone to latency, noise, degradation, and occasional failure under environmental or operational stress (Belgacem and Chihi, 2025). In SoS contexts, these issues are further compounded by dynamic membership and evolving configurations, as sub-systems may join, depart, or lose connectivity, interrupting the continuity of data exchange (Fang et al., 2020). Consequently, information within SoTS can be incomplete, inconsistent, or delayed.

Traditionally, DTs use Complex Event Processing (CEP) to reason over incoming

data streams, allowing them to detect patterns and trigger context-specific responses (Marah et al., 2025). Extending CEP to SoTS would enable higher-level reasoning across distributed, interdependent twins. However, conventional CEP engines assume deterministic and complete event flows (Akila et al., 2016). In practice, these assumptions rarely hold in SoTS deployments, where event arrival is asynchronous, uncertain, and prone to loss. Under such conditions, standard CEP frameworks risk producing unreliable or misleading outcomes.

These challenges highlight the need for an uncertainty-aware approach to event stream processing, one capable of maintaining reliability and continuity despite imperfect data. Such an approach should integrate predictive estimation directly within the processing pipeline, allowing uncertainty to be explicitly represented, propagated, and managed during event detection and reasoning. In doing so, it would enable more trustworthy decision-making under real-world SoTS conditions, where perfect data availability can rarely be ensured.

1.4 Problem Statement

SoTS are increasingly used to monitor and optimize the operation of critical infrastructures. However, the reliability of SoTS-level reasoning is hindered by data uncertainty. Due to the autonomy and heterogeneity of constituent twins, data produced across the system may be missing or inconsistent, leading to incomplete or misleading inputs for collective decision-making. Traditional CEP systems, which assume deterministic and complete data flows, are therefore ill-suited for SoTS environments, where real-world conditions and dynamic configurations disrupt data continuity.

The core problem addressed in this research is the absence of an uncertainty-aware approach for reliable and continuous CEP reasoning in SoTS, where incomplete data undermine the trustworthiness of system-level decisions.

Specifically, this research investigates how uncertainty manifests and propagates across constituent Digital Twins and how CEP mechanisms can be extended to maintain reasoning reliability under such conditions. The ultimate goal is to improve the continuity and trustworthiness of SoTS-level decision-making in real-world settings.

To achieve this, the following objectives are defined:

1. Conduct a systematic study of SoTS literature to identify existing gaps related to reliability, data continuity, and uncertainty management.
2. Formalize the types and sources of uncertainty affecting distributed event streams in SoTS, and analyze how they propagate to higher-level reasoning.
3. Design an uncertainty-aware CEP architecture incorporating mechanisms for data imputation, stream reconstruction, and probabilistic event handling.
4. Implement and evaluate the proposed framework using realistic, sensor-based scenarios that emulate SoTS conditions.

1.4.1 Research Questions

This study is guided by the following research questions:

1. **RQ1:** What are the main gaps and challenges in current SoTS research with respect to data reliability and uncertainty?

2. **RQ2:** How does uncertainty in distributed DT event streams propagate to SoTS-level decision-making?
3. **RQ3:** What architectural principles and mechanisms are required to make CEP engines uncertainty-aware in the context of SoTS?
4. **RQ4:** How effective is the proposed uncertainty-aware CEP framework in improving data reliability and decision quality compared to existing approaches?

1.5 Contributions

This thesis makes the following key contributions:

1. **A systematic study of Systems of Twinned Systems (SoTS):** A comprehensive literature review that classifies the state of SoTS research, identifying critical reliability and uncertainty gaps in current approaches.
2. **An uncertainty-aware CEP architecture:** The development of a distributed event processing system that integrates confidence estimation directly into event reasoning, enabling reliable detection and decision-making under incomplete data conditions.
3. **An evaluation and benchmarking framework:** A quantitative assessment of the proposed framework using a realistic SoTS simulated scenarios, demonstrating improved reliability and event detection under uncertain conditions.

1.6 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** introduces the theoretical foundations of Digital Twins, Systems of Systems, and Complex Event Processing, highlighting their convergence in the context of SoTS.
- **Chapter 3** presents a systematic review of SoTS literature, emphasizing existing approaches to reliability, data continuity, and uncertainty.
- **Chapter ??** formalizes the concept of uncertainty in SoTS event streams and analyzes its impact on event detection and reasoning.
- **Chapter 4** details the design of the proposed uncertainty-aware CEP framework, including the reconstruction, imputation, and probabilistic inference modules.
- **Chapter 5** describes the experimental setup, datasets, metrics, and results of the framework’s evaluation.
- **Chapter 6** concludes the thesis, summarizing findings, limitations, and directions for future work.

Chapter 2

Background

2.1 Digital Twins

2.1.1 Definitions

A digital twin is a high-fidelity virtual representation of a physical asset or system. It maintains a continuous bi-directional connection with its physical counterpart to enable real-time synchronization, calibration, and actuation (David and Bork, 2024). Across definitions, the DT represents an advanced form of cyber-physical integration in which digital and physical entities co-evolve through data exchange and feedback loops (Tao et al., 2018; Kritzinger et al., 2018).

Definitions for DTs converge on a few core characteristics. First, a DT serves as a high-fidelity virtual counterpart to a physical system, designed to replicate its structure, state, and behaviour with a high degree of accuracy. Second, it relies on bi-directional communication between the physical and digital domains, ensuring that changes in one are reflected in the other through continuous synchronization

and control feedback. Third, it depends on persistent data flow, achieved through IoT-enabled infrastructures, that keep the virtual model temporally aligned with its physical source.

From a data exchange perspective, Kritzinger et al. (2018) distinguish between three levels of representation in Digital Twin systems: the digital model, the digital shadow, and the digital twin. These levels describe the degree of integration between the physical and digital entities. A digital twin supports full bi-directional interaction, allowing it to both observe and actuate on its physical counterpart in real time. A digital shadow maintains a unidirectional connection in which information from the physical system is transmitted to the digital representation. This allows the digital model to reflect the current state of the asset but not to influence it. In contrast, a digital model is a static virtual representation that has no direct connection to the physical system.

2.1.2 Historical Evolution

The predecessor concept of using a “twin” for verification and testing originated in the aerospace domain. During NASA’s Apollo program in the 1960s, identical spacecraft were constructed so that engineers could mirror mission conditions on Earth and test alternative procedures in real time as flight data were received (Boschert and Rosen, 2016). This practice represented an early instance of using a physical counterpart to replicate operational conditions for simulation and fault diagnosis. A similar approach was later adopted in the aviation industry through “Iron Bird” test rigs, which physically replicated aircraft subsystems to validate system integration and control logic prior to flight (Boschert and Rosen, 2016). Together, these examples illustrate

how the motivation to experiment with counterparts before acting on the real system has long been embedded in engineering practice, laying the conceptual foundation for modern data-driven digital twins.

The formal concept of the digital twin emerged in the early 2000s, building on earlier notions of digital mirroring. Gelernter (1993) envisioned “mirror worlds” in which digital representations continuously reflected their physical counterparts, anticipating the synchronization principles later realized in digital twins. In the early 2000s, Grieves (2005) introduced the digital twin within the context of product life-cycle management (PLM), emphasizing the bidirectional coupling between physical systems and virtual models to enable continuous alignment throughout design, manufacturing, and operation. Around the same time, Hribernik et al. (2006) proposed the concept of a “product avatar,” linking digital representations to lifecycle integration through metadata and interoperability. NASA further refined the concept for aerospace applications, defining a digital twin as a high-fidelity, probabilistic simulation system capable of predicting system behaviour, performance degradation, and maintenance requirements across an asset’s lifecycle (Shafto et al., 2010; Glaessgen and Stargel, 2012).

With the rise of Industry 4.0, digital twins emerged as a key enabler of cyber-physical integration. By linking physical systems, sensors, and machines through cloud and IoT infrastructures, digital twins enable real-time analytics and optimization (Liu et al., 2021). Industrial platforms such as GE’s *Predix* (GE Vernova, 2024) and Siemens’ *MindSphere* (Siemens, 2025) exemplified this transition from theory to large-scale implementation, creating continuously updated digital counterparts for industrial assets. In academia, research paralleled these developments by exploring

digital twin reference architectures, interoperability standards, and domain-specific implementations across healthcare, transportation, and smart cities (Barricelli et al., 2019; Botín-Sanabria et al., 2022).

Overall, the evolution of the digital twin reflects a progression from simulation-based design tools toward data-driven systems that integrate sensing, computation, and actuation. Each stage of this progression—from early aerospace simulations to Industry 4.0—has increased the degree of automation, fidelity, and intelligence achievable within digital representations.

2.1.3 Architectures

Digital twin architectures describe how the physical and digital domains are coupled through data, models, and services. They provide a conceptual framework for defining the components, data flows, and feedback mechanisms. As DT implementations expanded across domains, several reference models and standards emerged to formalize their structure, functionality, and interoperability.

Tao et al. (2018) proposed one of the most influential architectural models, defining the digital twin as a five-dimensional construct composed of the physical entity, virtual entity, connection, data, and services. This framework emphasizes the iterative feedback loops between the physical and virtual entities, where data flow continuously through the connection and data dimensions to support real-time synchronization, simulation, and optimization. This model highlights the DT as a closed information loop in which the physical and virtual components co-evolve through data-driven feedback.

International bodies have formalized DT architectures through initiatives such as

ISO 23247 (ISO, 2021) and RAMI 4.0 (DIN, 2016). ISO 23247 defines a reference framework for DTs within manufacturing, specifying roles, entities, and data flows across four hierarchical layers. RAMI 4.0 similarly organizes industrial systems along three axes: hierarchical levels, lifecycle stages, and interoperability layers. This multi-dimensional structure connects DT functionality with broader Industry 4.0 principles. Together, these standards establish a shared vocabulary and design foundation for scalable, interoperable DT implementations across sectors.

2.2 Systems of Systems (SoS)

2.3 Complex Event Processing

Chapter 3

Literature Review

3.1 Systems of Twinned Systems (SoTS)

Systems of twinned systems represent the convergence of system of systems and digital twins. While SoS research emphasizes the orchestration of independent, heterogeneous systems to achieve collective goals (Maier, 1998), DTs focus on establishing a synchronized digital representation of physical assets. This enables real-time monitoring, analysis, and control (Kritzinger et al., 2018; Tao et al., 2018). Integrating these paradigms creates opportunities to extend both. DTs gain scalability and coordination across multiple interacting systems, while SoS benefit from intelligent, data-driven representations of their constituents.

In a SoTS, each constituent system is augmented with one or more digital twins that provide predictive, diagnostic, and prescriptive capabilities. These DT-enhanced constituents interoperate within a larger SoS enabling higher-level coordination, adaptation, and optimization across domains. This integration supports emergent capabilities, such as federated simulation, collaborative prediction, and coordinated

decision-making under uncertainty, that neither standalone DTs nor traditional SoS can achieve alone. As cyber-physical infrastructures grow in complexity, the SoTS vision has gained traction as a means to enhance reliability, interoperability, and adaptability. Recent research shows that combining DTs and SoS improves cross-system optimization and situational awareness in areas such as manufacturing, transportation, and critical infrastructure management (David and Bork, 2024; Ferreira et al., 2024), positioning SoTS as a scalable and adaptive evolution of both paradigms.

To explore this convergence, a systematic study was conducted to identify, classify, and analyze works at the intersection for SoTS research. This review aimed to answer the following high-level questions:

- **RQ1–RQ2: Conceptual Motivation and Integration.** *Why and how are DT and SoS combined?* These questions investigate the motivations, domains, and architectures that drive SoTS development, as well as the organizational and technical mechanisms by which these systems are composed.
- **RQ3–RQ4: Technical Characterization.** *What are the defining features of DTs and SoS within SoTS?* These questions examine autonomy, data exchange, modelling formalisms, and emergent behaviours across constituent systems.
- **RQ5–RQ7: Reliability, Maturity, and Implementation.** *How are non-functional properties such as reliability and security addressed? What is the current level of research and technical maturity? What technologies and platforms are most frequently used?*

Together, these questions guided the extraction of key patterns, challenges, and

design practices observed in existing SoTS literature. The study followed a systematic protocol consistent with the guidelines of Kitchenham and Charters (2007) and Wohlin et al. (2020), comprising the following phases:

1. **Database Search and Screening:** A comprehensive search across Scopus, Web of Science, IEEE Xplore, and ACM Digital Library, returning an initial corpus of 317 studies.
2. **Selection and Quality Assessment:** Application of explicit inclusion and exclusion criteria, duplicate removal, and quality evaluation
3. **Snowballing and Validation:** Forward and backward snowballing added relevant works, resulting in a final set of 80 high-quality primary studies.

3.2 Study Design

a systematic literature review was conducted following the established guidelines of Kitchenham and Charters (2007) and Wohlin et al. (2020). The review process was designed to capture both the conceptual and technical aspects of SoTS, focusing on how digital twin and system of systems principles are integrated, implemented, and evaluated in the literature.

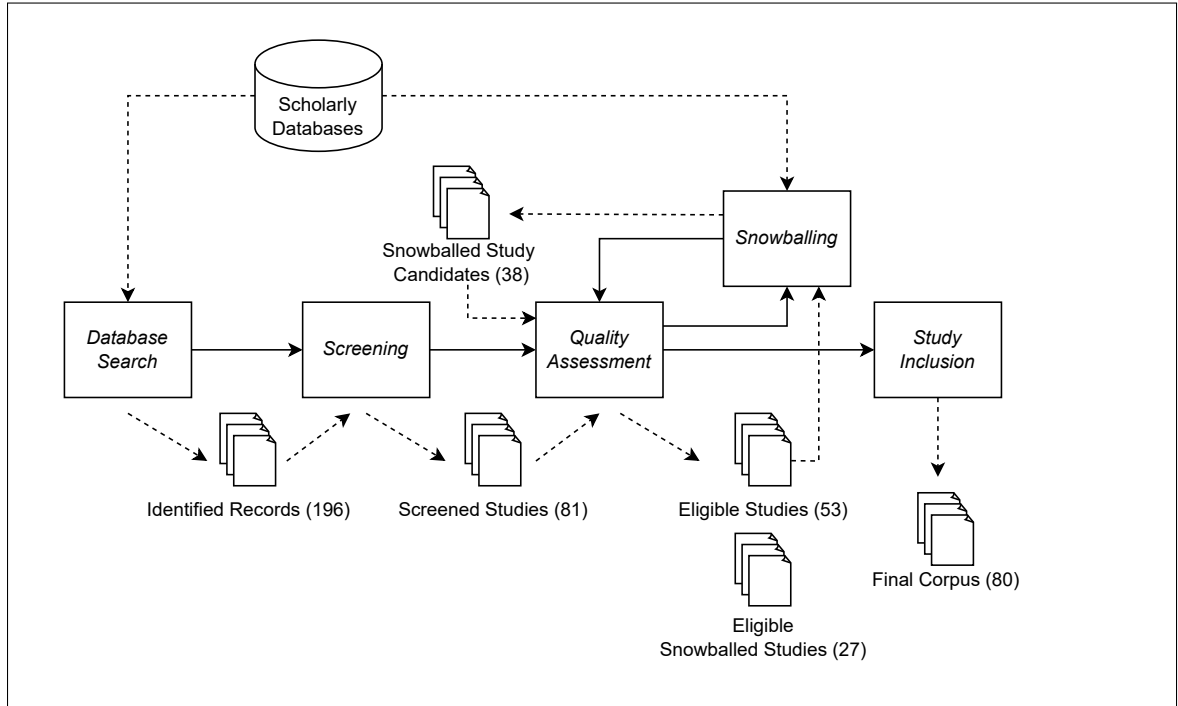


Figure 3.1: Overview of the study design and selection workflow used for the SoTS systematic literature review.

The study was conducted in four main phases, as illustrated in Figure 3.1:

1. **Planning and Scope Definition:** The review was structured around the research questions introduced in Section 3, which investigate the motivations, architectures, reliability concerns, maturity, and technological underpinnings of SoTS. A search protocol and quality assessment strategy were developed prior to data collection to ensure transparency and repeatability.
2. **Database Search and Screening:** A structured query was executed across four major indexing databases—*Scopus*, *Web of Science*, *IEEE Xplore*, and *ACM Digital Library*. Searches were limited to peer-reviewed publications

within Computer Science and Engineering disciplines, and included synonymous expressions for SoTS such as “aggregated digital twins,” “system of digital twins,” and “digital twin of systems.”

3. **Selection and Quality Assessment:** The initial corpus was screened through automated and manual filtering to remove duplicates and out-of-scope papers. Each remaining study was independently assessed by two reviewers against predefined exclusion criteria (e.g., relevance, completeness, and clarity of SoS/DT descriptions). Inter-rater agreement and Cohen’s κ were computed to verify the consistency of inclusion decisions.
4. **Data Extraction and Synthesis:** Data were extracted using a structured coding framework, focusing on: (1) conceptual alignment between DT and SoS principles, (2) system architectures and modelling approaches, (3) reliability and non-functional considerations, (4) technical and research maturity, and (5) implementation technologies. A combination of quantitative (frequency-based) and qualitative (content analysis) synthesis methods was used to identify recurring themes and patterns.

The search and selection statistics are summarized in Table ???. A total of 317 studies were initially retrieved, and after the application of inclusion, exclusion, and quality filters, **80 primary studies** were included in the final corpus. These studies collectively represent the current state of SoTS research, capturing both theoretical advances and empirical applications across diverse domains such as manufacturing, energy, and urban infrastructure.

Table 3.1: Search statistics

Search round	All	Excluded	Included	Agreement
Automated search				
↳ Duplicate removal	317	121	196	<i>IRA</i> = 0.88 / κ = 0.734
↳ E0	196	71	125	
↳ E1–E3	125	44	81	
Quality assessment				
↳ Q1 or Q2 insufficient	81	22	59	<i>IRA</i> = 0.944 / κ = 0.488
↳ Q3 insufficient	59	6	53	
↳ Q4 insufficient	53	0	53	
Subtotal	317	264	→53 (16.72%)	
Snowballing				
↳ Backward				<i>IRA</i> = 0.944 / κ = 0.488
↳ Selection by reference	1 666	1 521	145	
↳ Selection by abstract	145	132	13	
↳ Forward	586	561	25	
↳ Total abstracts screened	731	693	38	
Quality assessment				
↳ Q1 or Q2 insufficient	38	9	29	<i>IRA</i> = 0.944 / κ = 0.488
↳ Q3 insufficient	29	2	27	
↳ Q4 insufficient	27	0	27	
Subtotal	2 252	2 225	→27 (1.19%)	
Total	2 569	2 489	→80 (3.11%)	

3.3 Survey of Systems of Twinned Systems

This section reviews existing studies on Systems of Twinned Systems (SoTS).

3.3.1 SoTS Types

- Directed, acknowledged, collaborative, and virtual SoTS. - Examples of studies for each type (quality threshold: 8).

3.3.2 Trends in SoTS Research

- Rapid growth in acknowledged SoTS. - Emerging but smaller growth in collaborative and virtual SoTS. - Complexity increases across types; reliability may be a limiting factor for higher-level SoTS. - Very few studies perform formal validation of reliability.

3.4 Reliability Mechanisms in SoTS

3.4.1 Maturity Levels of Reliability Consideration

- Not addressed, mentioned only, architecturally addressed, explicitly modeled, validated.

3.4.2 Dimensions of Reliability

- Constituent reliability. - Data and communication reliability. - Coordination reliability. - Emergence reliability. - Runtime adaptation reliability.

3.4.3 Mechanism Categories

- Preventative. - Fault-tolerant. - Detective. - Corrective.

3.4.4 Mapping Reliability to SoTS Types

- Unique reliability concerns for directed, acknowledged, collaborative, and virtual SoTS. - Positioning of this study in the data and communication dimension, with implications for other dimensions.

3.5 Complex Event Processing in DT and SoS Contexts

- Use of CEP with digital twins. - Use of CEP with systems of systems. - Observation: existing production-ready CEP engines do not address reliability.

3.6 Research Gap and Contribution

3.6.1 Gap

- SoTS is growing rapidly, but implementations rarely consider reliability. - CEP is well-studied in DTs but absent in SoTS due to missing-data and reliability challenges.

3.6.2 Contribution

- This thesis ensures CEP can be applied in SoTS by introducing a reliability-aware imputation mechanism for streaming data.

Chapter 4

Architecture

This chapter presents the architecture of the proposed framework designed to manage uncertainty in SoTS through real-time data reconstruction, confidence propagation, and event reasoning.

4.0.1 Architectural Objectives

The framework architecture is guided by two primary objectives that ensure reliable, interpretable, and interoperable operation across distributed SoTS constituents:

- **Data Continuity:** Preserve uninterrupted information flow within the SoTS by detecting and reconstructing missing data, ensuring that downstream reasoning operates on complete event streams.
- **Uncertainty Transparency:** Explicitly encode the reliability of reconstructed information as confidence values attached to each event, ensuring consistent handling of uncertainty throughout the processing pipeline.

Design Principles

We follow these guiding principles to meet the architectural objectives defined:

- **Continuity under data loss:** Reconstruct missing events to maintain uninterrupted information flow
- **Transparency of uncertainty:** Represent reconstruction reliability explicitly through a `confidence` attribute attached to each event, allowing uncertainty to remain visible.
- **Loose coupling and modularity:** Use a publish–subscribe communication model to decouple data producers and consumers, enabling independent operation and flexible integration.
- **Separation of concerns:** Clearly differentiate the roles of data generation, reconstruction, coordination, and event detection to improve maintainability and extensibility.
- **Scalability and reusability:** Integrate heterogeneous components, streams, predictors, messaging clients, and CEP adapters, through a uniform interface and pluggable configuration design.

These objectives address a core limitation of existing SoTS and CEP infrastructures, which often assume complete data streams. By integrating reconstruction and confidence tracking directly into the shared event pipeline, the framework replaces absence with approximation. Allowing information to remain accessible even under uncertainty.

4.1 Overview

The proposed framework adopts a layered, message-driven architecture that enables both autonomy and cooperation among DT constituents within a SoTS. Each DT operates as an independent node capable of reconstructing sensor data, propagating confidence, and performing local event reasoning while remaining connected to a shared asynchronous communication layer.

As shown in Figure 4.1, the framework follows a publish subscribe model: each DT publishes data from its own sensors to a shared event stream and may subscribe selectively to datastreams from other constituents. This preserves data ownership while allowing situational awareness to emerge through controlled information exchange. Each DT hosts a local instance of the SoTS framework, configured by a researcher to define subscriptions, reconstruction parameters, and complex event patterns. Through this mechanism, DTs can collaborate and access system-level information without relying on centralized coordination.

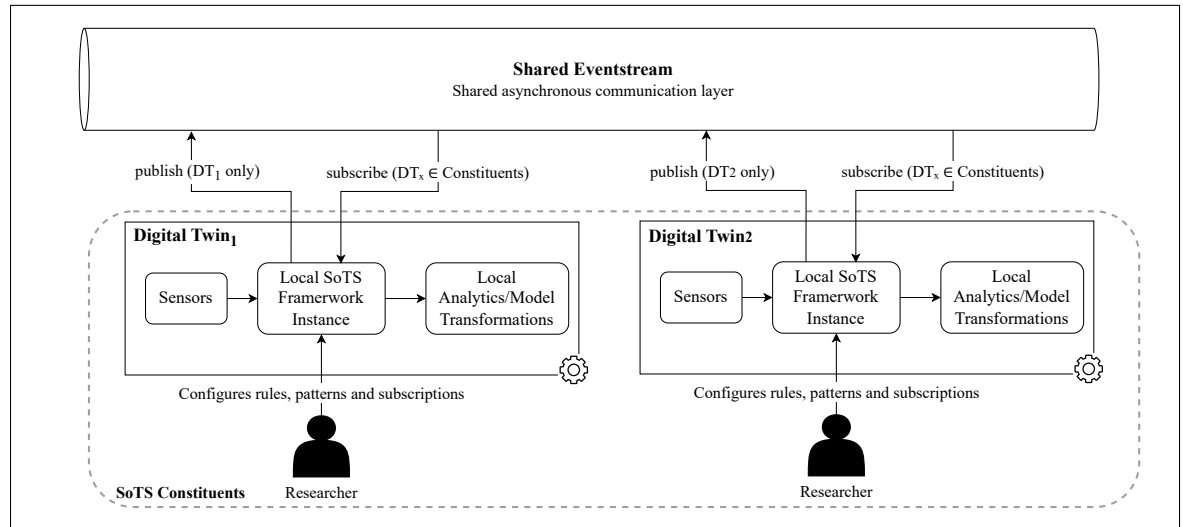


Figure 4.1: Context diagram

4.1.1 High-Level Pipeline Description

Figure 4.2 presents the high-level architecture of the proposed framework. The system is organized into two primary layers: the Event Stream Layer, which provides the messaging infrastructure for asynchronous data exchange, and the Processing Layer, which hosts the active modules responsible for coordination, reconstruction, and reasoning. Solid arrows indicate event data flow, while dashed lines represent control signals.

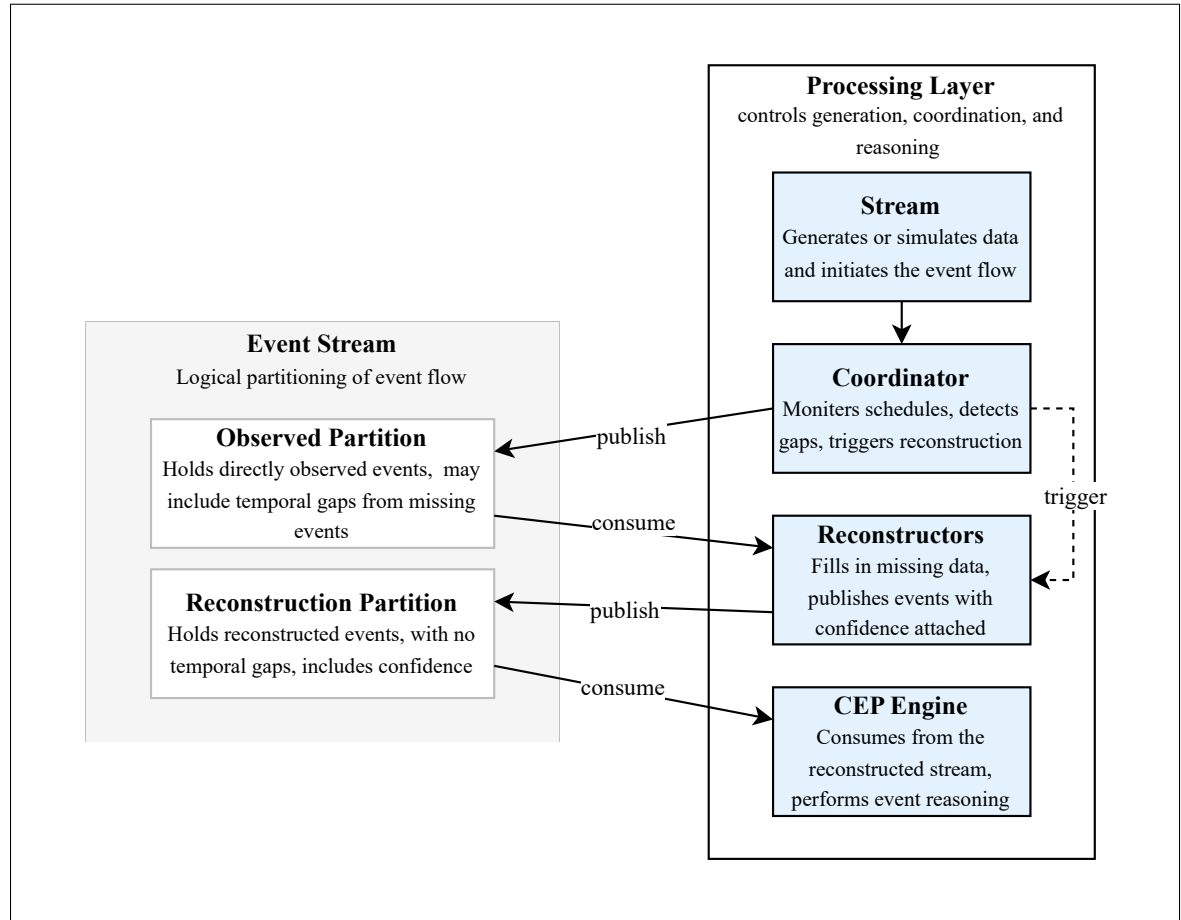


Figure 4.2: Container-level Diagram

At a high level, the framework operates as a continuous event pipeline that

transforms incomplete sensor data into a reconstructed, uncertainty-aware stream. **Streams** act as data generators, emitting observed events that may contain temporal gaps due to missing readings. The **Coordinator** manages event scheduling and serves as the control point for dissemination, publishing available sensor readings to the observed stream or invoking the **Reconstructor** when expected events are absent. The *Reconstructor* consumes all observed data and appends confidence metadata to each event. It continuously updates the underlying predictors and generates synthetic events when required, producing a unified stream that merges both observed and reconstructed data. The resulting reconstructed stream provides a temporally complete, confidence-annotated data flow for downstream reasoning by the **CEP Engine**. This design ensures that every event follows a consistent lifecycle from generation to interpretation, maintaining continuity and traceability across the entire pipeline.

1. **Streams** generate sensor data and forward them to the Coordinator.
2. The **Coordinator** schedules event publication, detects missing readings, and triggers reconstruction when gaps are found.
3. The **Reconstructor** consumes observed events, appends confidence information, and imputes missing values when invoked.
4. All processed events are published to the *reconstructed stream*, forming a continuous, confidence-annotated data flow.
5. The **CEP Engine** consumes the reconstructed stream for local or global pattern detection.

4.2 Framework Architecture

The framework is organized into three primary functional layers: Coordination, Reconstruction, and CEP; supported by a Messaging Layer.

4.2.1 Coordination Layer

The Coordination layer governs the scheduling and synchronization of data collection across multiple event streams. Each stream corresponds to an initialized data source, which may be simulated, randomly generated, or connected to live sensor inputs.

Core Function. The Coordinator advances according to a fixed tick-based schedule that defines the expected arrival time of events. At every tick, it polls the associated stream for a new reading. If a measurement is received within the allowed interval, it is published as an observed event. If no data are received before the timeout, the Coordinator detects a missing observation and issues a reconstruction request to the Reconstructor. Once the reconstructed event is produced, it is reintegrated into the reconstructed stream, ensuring that the resulting output remains temporally complete and consistent. Through this mechanism, the Coordinator provides time alignment, while handling dropped or delayed sensor updates.

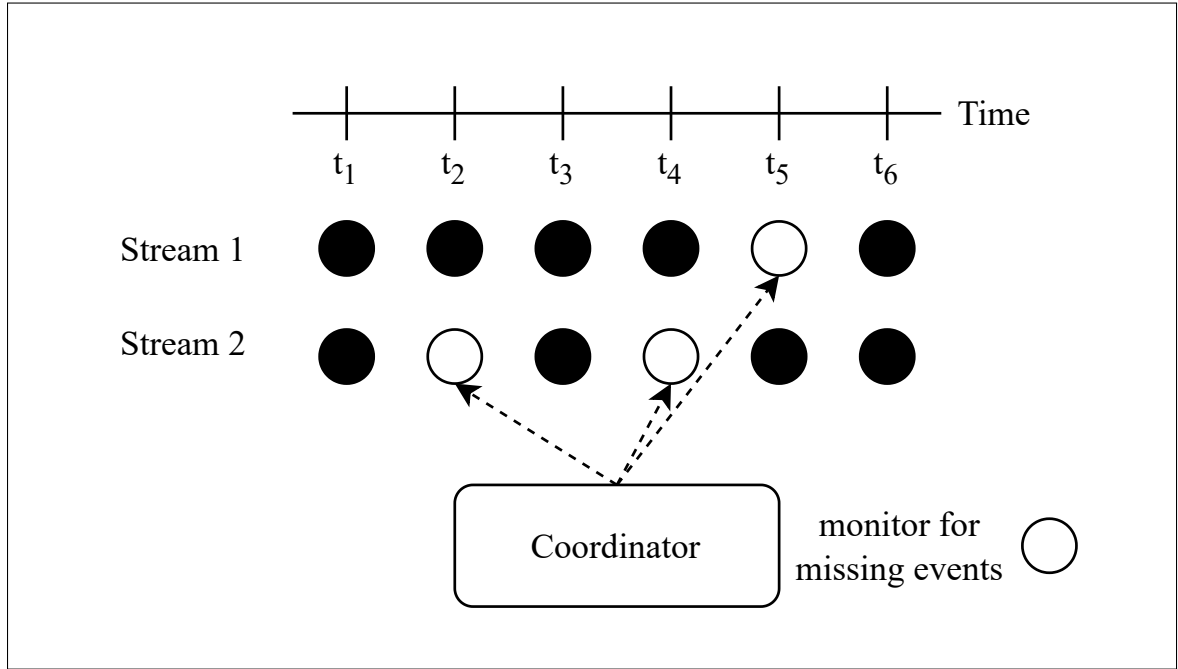


Figure 4.3: Coordinator layer performing scheduling and gap detection.

4.2.2 Reconstruction Layer

The Reconstruction layer preserves data continuity through statistical estimation. Its primary role is to infer missing sensor readings and quantify the reliability of each estimate. Each Digital Twin hosts its own Reconstructor instance, which maintains a continuously updated predictor that models local sensor dynamics.

Core Function. When the Coordinator detects a missing event, it invokes the Reconstructor to estimate the absent reading. The predictor is model-agnostic, any statistical or machine learning technique can be employed, but in practice, a *Kalman*

filter is used as the default estimator due to its recursive structure and built-in uncertainty propagation. The filter maintains a local state model of the sensor process:

$$x_t = Ax_{t-1} + Bu_t + w_t, \quad z_t = Hx_t + v_t$$

where w_t and v_t denote process and measurement noise, respectively. For each reconstruction, the filter outputs both an estimated state $\hat{x}_t$ and an associated confidence score $c_t \in [0, 1]$, derived directly from the posterior covariance or normalized residuals.

Each reconstructed event is emitted in the form:

$$\langle id, t, \hat{x}, c, s = \text{reconstructed} \rangle$$

and published to the **reconstructed** stream, ensuring that the output stream remains temporally complete and uncertainty-aware for subsequent processing.

Design Rationale. Kalman filters provide adaptive, noise-tolerant estimation well suited to continuous, partially observed data streams. Because covariance is an intrinsic part of the filter’s update mechanism, confidence values are obtained directly rather than computed through an external calibration process. This design supports model transparency and allows downstream components—such as the CEP Engine—to reason over reconstructed data using confidence as a compact representation of uncertainty.

4.2.3 Complex Event Processing (CEP) Layer

The *Complex Event Processing (CEP)* layer performs higher-level reasoning over the unified event stream. It evaluates temporal patterns, correlation rules, and aggregation functions to derive composite events that represent system-level behaviors or anomalies. The layer is implemented using the Esper engine and interfaces with the messaging system through a lightweight Python adapter.

Core Function. Observed and reconstructed events are serialized in JSON format and streamed into the CEP engine, where event queries are defined using Esper’s Event Processing Language (EPL). These queries specify conditions and temporal windows for event detection, for example:

```
SELECT AVG(value) AS avgTemp
FROM TemperatureStream[range 10 sec]
WHERE confidence > 0.7;
```

Each incoming event carries a confidence value that quantifies the reliability of its measurement or reconstruction. These confidence attributes are preserved throughout rule evaluation and can be aggregated or filtered based on user-defined policies. As a result, the CEP layer produces complex events annotated with interpretable confidence levels, enabling uncertainty-aware reasoning and reliable downstream analytics.

4.3 Messaging System

At the core of the framework lies an abstracted event-streaming channel that enables all events to exist within a shared communication space before being consumed by other layers. While the current implementation uses ZeroMQ zer ([n. d.]), the design remains transport-agnostic and compatible with other publish–subscribe middleware supporting topic-based routing, such as MQTT OASIS Open (2014). This abstraction allows the messaging layer to serve as a reusable communication stream across deployments and enables constituent systems to interact through a unified event interface.

Common Event Plane and Topic Structure. All constituents communicate through a shared event plane—a logical message bus that captures the full lifecycle of events across the SoTS. Messages are partitioned by event type using a simple yet expressive topic hierarchy: The framework distinguishes between two primary event topics: The framework maintains two primary event topics:

- **observed.<stream_id>** — original sensor readings as emitted from the physical data sources.
- **reconstructed.<stream_id>** — a temporally complete stream that merges original observations with synthetically generated events introduced to fill missing data points, ensuring continuity across the timeline.

The **observed** topic thus holds only authentic readings, while the **reconstructed** namespace maintains a temporally complete, uncertainty-aware event stream. Because both follow a common event schema, modules can seamlessly switch between observed-only or fused data depending on their operational needs.

Prefix-Based Filtering and Routing. ZMQ’s built-in prefix-matching subscription enables efficient filtering and dynamic routing of messages. Modules may subscribe to topic prefixes (e.g., `observed.*`) to receive all observed streams without enumerating each explicitly. This mechanism reduces message duplication and allows flexible composition of event flows, making it particularly suited for dynamically evolving SoS environments where streams may appear or disappear at runtime.

Table 4.1: Topic hierarchy and routing semantics in the messaging system.

Topic Pattern	Description / Subscription Scope
<code>observed.<stream_id></code>	Original sensor readings emitted by a single stream (e.g., <code>observed.beach1</code>).
<code>observed.*</code>	All observed data across all streams; used by modules that require raw input.
<code>reconstructed.<stream_id></code>	Fused stream containing both observed and reconstructed values for one sensor.
<code>reconstructed.*</code>	All temporally complete, uncertainty-aware streams; typical input for the CEP layer.

Endpoints, Roles, and Visibility. The system follows a distributed client–server topology in which all participants share the same set of messaging endpoints, collectively forming the event bridge. Each module—whether publisher or subscriber—connects to this bridge, ensuring a consistent global view of the SoS data stream. Within this architecture, modules formally declare themselves as event generators (e.g., data

sources or reconstructors) or event consumers (e.g., CEP engines, loggers, or controllers). By maintaining the same endpoints, all participants operate within a synchronized event plane, meaning that any message broadcast by one constituent becomes immediately visible to all subscribers authorized for that topic. This design ensures transparency, reproducibility, and system-level situational awareness while preserving constituent autonomy: each DT determines which streams it publishes and consumes.

Extensibility and Modularity. Because all topics adhere to a unified schema, new DTs or analytical components can attach to the event plane dynamically without reconfiguration. This plug-and-play design allows the SoTS to evolve organically as new data sources, filters, or event-processing rules are introduced, sustaining interoperability and composability across the framework.

4.4 Event Lifecycle and System Flow

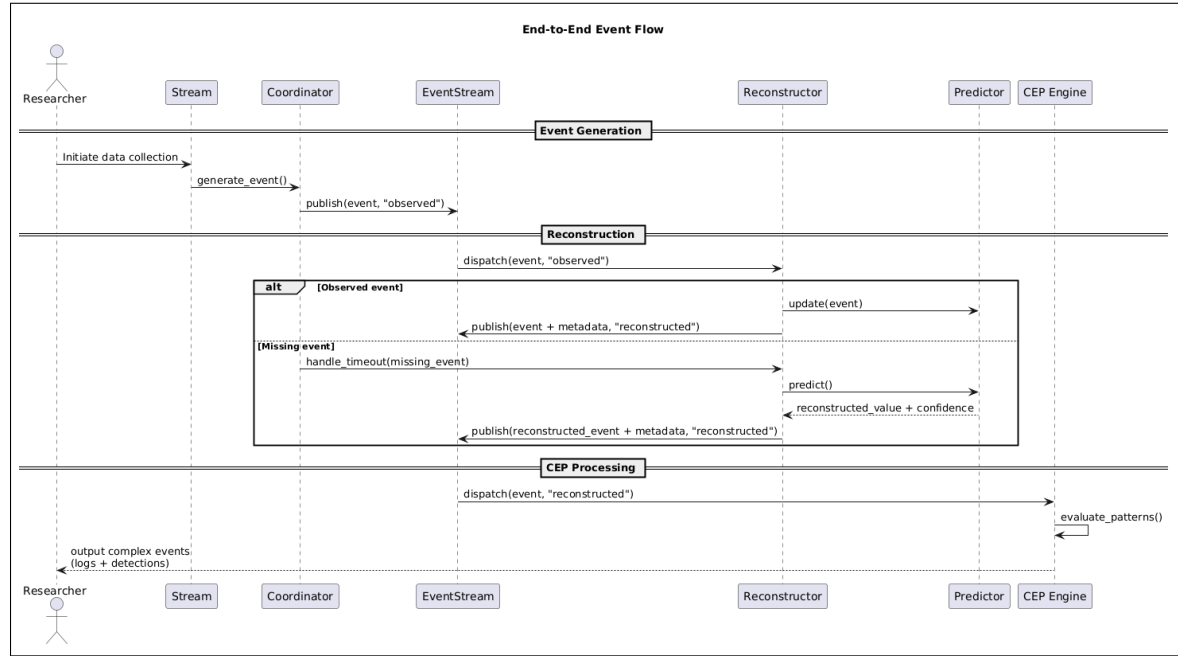


Figure 4.4: Flow of an event through the system

The sequence diagram (Figure 4.4) illustrates the interaction between the major components of the framework. The process occurs in three phases: **Event Generation**, **Reconstruction**, and **Complex Event Processing**.

(1) Event Generation

The *Researcher* initiates data collection through the *Stream* component. The *Coordinator* periodically invokes the `generate_event()` routine, converting sensor readings into standardized event objects. Each event is published to the *EventStream* under the "observed" topic, carrying the most recent real sensor values.

(2) Reconstruction

The *Reconstructor* continuously consumes events from the "observed" stream, using the real values to update the internal state of its predictors. Each event is then augmented with metadata derived from the predictors, such as confidence and the predicted value, and republished into the "reconstructed" stream. If a temporal gap is detected, the *Coordinator* directly invokes the *Reconstructor*, which synthesizes a missing event using the active predictor (e.g., Kalman filter). The resulting synthetic event is inserted into the reconstructed stream, ensuring that all timestamps are represented and no temporal gaps remain.

(3) Complex Event Processing

The *CEP Engine* subscribes to the "reconstructed" stream and applies pattern-matching rules to detect higher-level phenomena such as anomalies or behavioural trends. Each event carries confidence metadata that quantify the estimated reliability of its reconstructed values. These attributes are retained throughout the CEP evaluation process, enabling rules to compute, aggregate, and query confidence levels across correlated events. The resulting complex events are then logged for downstream analysis.

4.5 UML and Implementation

Figure 4.5 presents the UML diagram of the proposed framework.

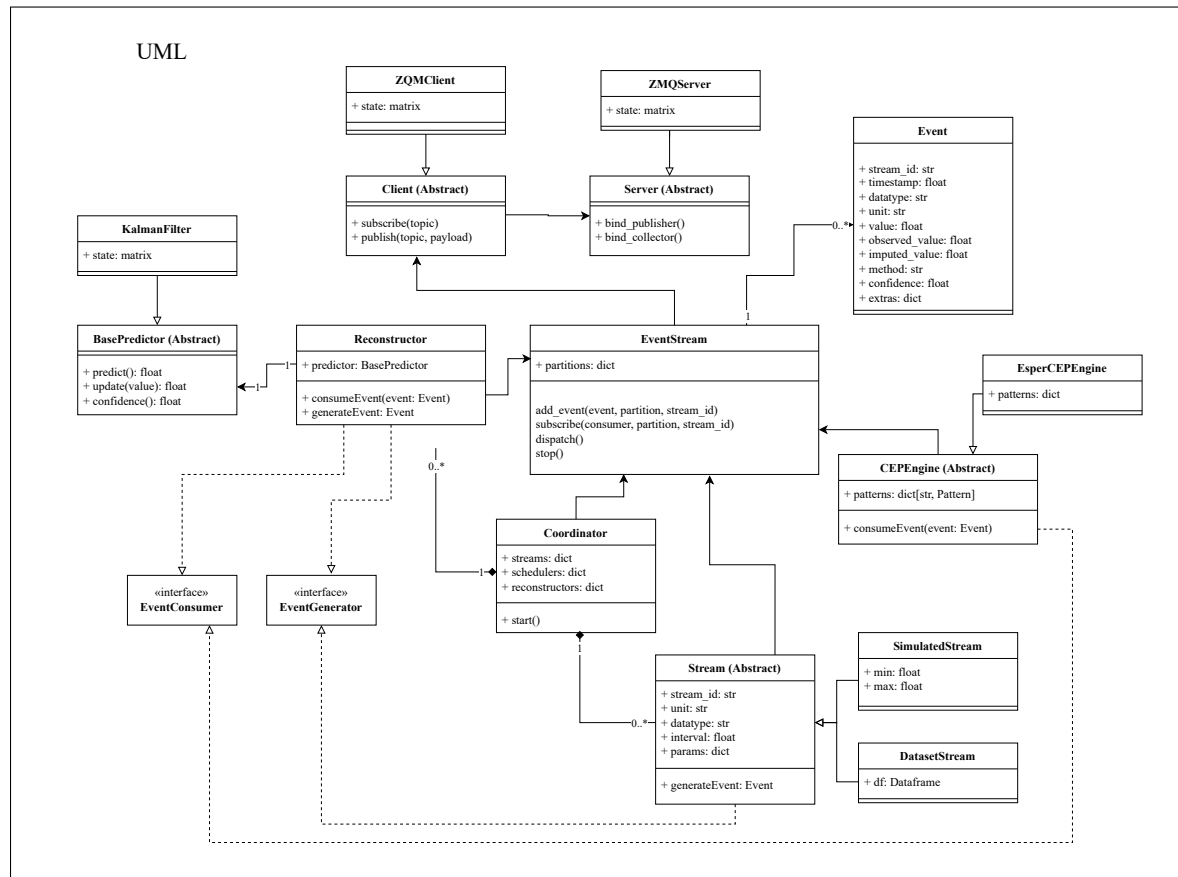


Figure 4.5: UML Class Diagram

4.5.1 Implementation Details

The framework is implemented primarily in **Python 3.10**, with a complementary **Java 21** component providing access to the Esper CEP Engine. The two environments communicate through **ZeroMQ** sockets using JSON-encoded event messages. Configuration files written in JSON define stream parameters, reconstruction models, and CEP patterns, allowing experiments to be modified or reproduced without altering the source code.

Runtime Integration. The Java process is launched automatically by the Python runtime when the CEP module is enabled, establishing an inter-process bridge between the two environments. Esper runs as an independent Java process, while stream generation, coordination, and reconstruction are handled entirely in Python. Events are serialized as JSON objects and exchanged through ZeroMQ sockets, providing a lightweight, transport-agnostic channel for cross-language communication.

Dependencies. **Python:** NumPy, SciPy, Pandas, PyZMQ, scikit-learn, pytest. **Java:** Esper 9.0.0, Gson, Jackson Databind, JeroMQ, Janino, Commons-Compiler, SLF4J.

CEP Integration. During execution, the Python *Reconstructor* publishes reconstructed events to a ZeroMQ topic subscribed to by the Java CEP engine. Incoming JSON-formatted messages are parsed and injected into the Esper runtime, where they are continuously evaluated against event-pattern rules defined in external configuration files. When a rule condition is satisfied, Esper emits a pattern match—annotated with aggregated confidence levels, which is logged for further analysis.

4.6 Scope and Limitations

The architecture currently assumes in-order event delivery. Late or out-of-order events are not reprocessed by the CEP engine, as this would require event-time semantics and rule retraction capabilities beyond the current scope. However, delayed readings may be used in future work for batch updates of predictive models, when late data arrives.

4.7 Summary

This chapter outlined the design and operation of the uncertainty-aware SoTS framework. It introduced the Reconstruction, Coordinator, and CEP layers, each implementing a core aspect of uncertainty management. The messaging model enables modular integration of independent constituents, supporting the SoTS principles of autonomy and composability. Together, these components realize a system capable of maintaining data continuity and transparent confidence propagation across heterogeneous event streams.

Chapter 5

Evaluation and Results

5.1 Evaluation Design

This chapter evaluates the performance and reliability of the proposed uncertainty-aware CEP framework under controlled missing data scenarios. The goal is to assess how well the system maintains event continuity when sensor streams are partially missing, reflecting real-world disruptions.

5.1.1 Evaluation Overview

The evaluation is structured around three key components:

1. **Missingness Patterns:** Controlled data loss patterns are introduced to simulate uncertainty and partial observability.
2. **Dataset and Scenario:** A real-world multi-sensor dataset is selected to represent a distributed cyber-physical environment.

3. **Event Patterns and Predictors:** Defined CEP rules and prediction models are tested to analyse system-level behaviour.

5.1.2 Missingness Patterns

To evaluate reconstruction robustness under uncertainty, the experiments introduce controlled data loss through a hybrid missingness scenario applied at three loss levels (10%, 20%, 30%). This scenario blends both random and structured missingness mechanisms to more realistically emulate disruptions observed in SoTS environments. Results are compared against two reference cases: an **Oracle** baseline (no missing data) and a **Structural Gap** baseline (irrecoverable data loss).

- **Hybrid Missingness (Proposed):** Combines two complementary mechanisms with equal probability: (i) *Random (MCAR)* removals, where individual readings are deleted independently to represent sporadic communication losses or packet drops; and (ii) *Block Missingness*, where consecutive readings (ranging from 2–6 hours) are removed to simulate short sensor outages or temporary twin disconnections.
- **Structural Gap (Baseline):** Represents long or permanent data absences due to known downtime, maintenance periods, or total node failure. These gaps are intentionally unreconstructed and serve as a lower-bound reference for assessing the of event detection under persistent loss.
- **Oracle (Baseline):** Ideal condition with complete, uninterrupted data, used as the upper-bound benchmark for reconstruction accuracy and event-detection confidence.

5.1.3 Dataset Description

The experiments use the Chicago Park District Beach Water and Weather Sensors dataset (Chicago Park District, 2025), which records hourly measurements for variables such as water temperature, turbidity, wave height, and wave period across multiple beaches along Lake Michigan. For this study, three representative sites are used: Calumet Beach, Montrose Beach, and 63rd Street Beach. Their geographical distribution along the shoreline (Figure 5.1) makes them well-suited for modelling a SoTS scenario.

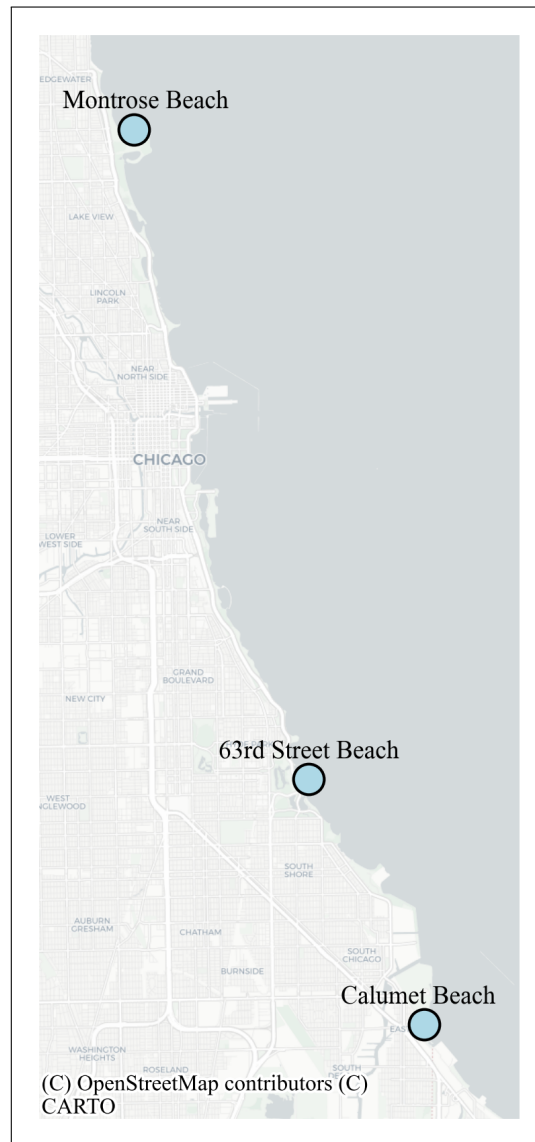


Figure 5.1: Map of beaches

Each beach is modelled as a DT node continuously monitoring local water and environmental conditions. Together, these nodes form a distributed network that collectively represents the coastal monitoring system for safety monitoring and environmental awareness

The beaches are located in adjacent regions, meaning that environmental changes,

such as wave propagation, temperature shifts, or turbidity gradients, at one location can influence conditions at others. This interdependence captures the *emergent behaviour* typical of SoS environments, where local phenomena have system-level impacts. In a realistic deployment, such a network could support autonomous actions, such as:

- issuing public safety alerts for high bacterial or turbidity levels,
- coordinating automated water pumping or drainage responses during high wave events,
- dynamically adjusting sensor sampling rates based on neighbouring conditions.

Thus, the dataset provides a strong foundation for evaluating the framework in a realistic SoTS context.

5.1.4 Dataset Statistics and Visual Overview

Figure 5.2 presents the per-beach distributions of the key measured attributes, water temperature, turbidity, wave height, and wave period, during the 2015 summer season. These distributions capture the statistical variability observed at each site and highlight local environmental differences across the Chicago coastline. While the overall shapes remain consistent across beaches, subtle shifts in means and variances reflect their location differences influenced by shoreline shape, wave exposure, and localized weather conditions.

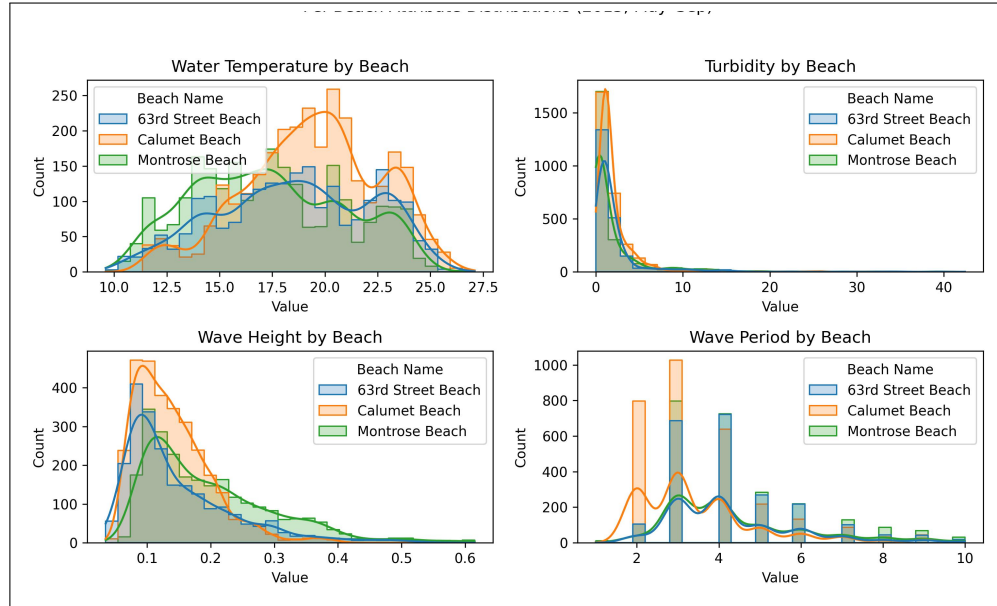


Figure 5.2: Per-beach attribute distributions (May–September 2015).

Figure 5.3 provides a time-series overview for each beach. Water temperature exhibits gradual seasonal warming, while turbidity and wave-related attributes show irregular bursts corresponding to storm events, rainfall, or inflow disturbances.



Figure 5.3: Hourly time-series overview for the three selected beaches (May–September 2015).

5.1.5 Scenario Setup

The experimental process follows the general workflow below:

- Each beach node generates sensor readings at fixed hourly intervals, representing continuous monitoring of environmental parameters.
- When data gaps occur, the system automatically estimates the missing values using the selected prediction model to maintain a consistent event flow.
- Both the original and reconstructed data streams are then used for complex event detection, allowing the evaluation of how uncertainty and data loss influence overall system behaviour.

5.1.6 CEP Evaluation and Event Patterns

To evaluate detection performance under uncertainty, the CEP layer was tested with varying confidence thresholds and hierarchical patterns.

Each experiment was conducted using detection cutoffs of **0.65**, **0.75**, **0.85**, and **0.95**, enabling comparison across multiple confidence levels. These thresholds represent progressively stricter detection requirements, allowing analysis of system performance under relaxed, moderate, and highly selective confidence conditions.

The evaluation uses hierarchical set of patterns, progressing from simple sensor-level conditions (atomic events) to beach-level (local complex) and system-wide (distributed complex) alerts. This structure reflects the SoTS concept: local twins operate autonomously but can trigger collective behaviour when multiple conditions align across nodes. For all complex patterns, event confidence is computed as the **mean confidence** of the constituent events that make them up.

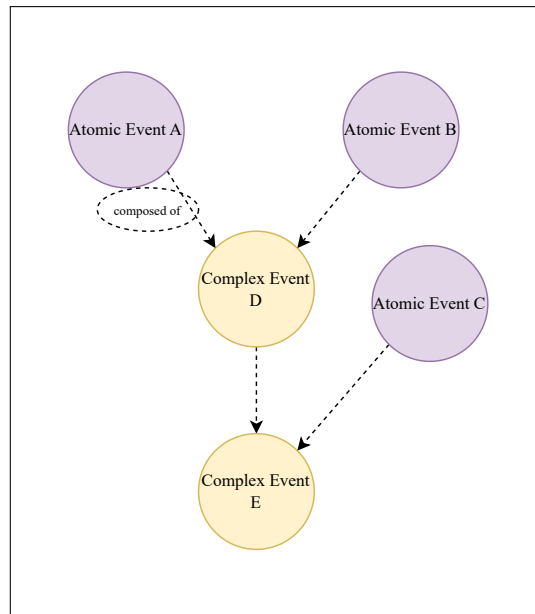


Figure 5.4: Illustration of a sample event composition of atomic and complex events

Table 5.1: Hierarchy of event patterns used in the evaluation.

Event Type	Example Name(s)	Condensed Description
Atomic (Sensor-level)	HighTurbidity, LowWaterTemp, HighWave, HighWavePeriod	Single-sensor threshold events per beach, such as turbidity > 8.5 NTU, wave height > 0.35 m, or temperature < 22°C. These serve as the building blocks for higher-level detection.
Local Complex (Beach-level)	LocalAlert	Combines two atomic events within the same beach (e.g., high turbidity and low temperature at Calumet, or high turbidity and high wave height at Montrose) occurring within 3 s. Represents short-term local correlations indicative of abnormal conditions.
Distributed Complex (Cross-beach)	AlertSpread, TripleAlert	Captures inter-beach propagation of alerts (e.g., Montrose → Calumet → 63rd) within short windows (5–7 s). Reflects emergent regional SoTS behaviour, where local anomalies align across multiple sites.

5.1.7 Predictor Configuration

Two predictors were chosen to evaluate the reconstruction of the missing sensor readings: the Kalman Filter (KF) and the Particle Filter (PF).

Kalman Filter. The Kalman Filter is a recursive state estimator that models the evolution of a variable as a linear dynamical system with Gaussian noise. At each step, the filter predicts the next state based on the previous estimate and corrects it when a new measurement becomes available, balancing process and measurement uncertainty through their respective covariances.

Confidence is computed using the covariance trace, which quantifies how the current uncertainty compares to the filter’s initial uncertainty. It is derived directly from the posterior covariance matrix of the Kalman filter and defined as:

$$\text{confidence}_{\text{KF}} = 1 - \frac{\text{trace}(P_k)}{\text{trace}(P_0)} \quad (5.1.1)$$

where P_k denotes the posterior covariance matrix at the current time step, and P_0 represents the initial covariance matrix used for normalization. A lower trace value indicates reduced overall uncertainty, leading to higher confidence, whereas an increasing trace reflects the accumulation of uncertainty over time.

Rational The confidence formula used is derived directly from the Kalman filter’s posterior covariance, which represents the estimator’s internal measure of uncertainty. Minimizing this quantity is equivalent to minimizing the mean-squared estimation error, making it a good basis for quantifying estimator precision. Building on the analysis of Kang and Xu (2021), who characterize the magnitude and decay of the Kalman filter’s error covariance as quantitative descriptors of uncertainty, the normalized covariance trace captures the overall reduction of uncertainty as the filter converges.

Particle Filter - The Particle Filter approximates the posterior distribution through

a set of weighted samples (particles) that evolve under the process model. Each particle represents a potential system state, with weights updated when new measurements arrive. When data are missing, particles propagate without correction. The Particle Filter used in this work follows the Sampling Importance Resampling (SIR) described by Arulampalam et al. (2002).

Confidence is computed from the spread of particles:

$$\text{confidence}_{PF} = \frac{1}{1 + \text{Var}(\mathbf{p})}$$

where $\mathbf{p} = \{p_1, p_2, \dots, p_N\}$ denotes the set of particles representing possible realizations of the system state. The variance of this distribution reflects the uncertainty in the estimated state: a tightly clustered particle set indicates high confidence, while a wide spread corresponds to greater uncertainty.

Rational The confidence formula used ...

Configuration and Initialization

All predictors were tuned through a grid search using data from a subset of the dataset, Montrose Beach dataset (2016), which was excluded from subsequent evaluation. The tuning process simulated a baseline of 30% missingness under an MCAR pattern to represent moderate uncertainty conditions. Parameter selection was guided by minimizing the Root Mean Squared Error (RMSE) between reconstructed and observed values, while simultaneously promoting a negative correlation between reconstruction error and model confidence. In other words, low-accuracy predictions were expected to return lower confidence, and high-accuracy predictions higher confidence.

A combined score function was used to balance these two criteria:

$$S = \alpha \cdot \text{RMSE} + (1 - \alpha) \cdot (1 - \rho), \quad (5.1.2)$$

where ρ denotes the Pearson correlation between the model’s confidence estimates c_t and the corresponding reconstruction errors $|y_t - \hat{y}_t|$. The weighting factor $\alpha = 0.75$ prioritizes reconstruction accuracy while still rewarding consistent confidence outputs.

Table 5.2: Grid search parameter ranges for predictor tuning (Montrose Beach, 2016).

Predictor	Parameter	Range Tested
Kalman Filter	Process noise Q	[0.001, 0.005, 0.01, 0.05, 0.1, 0.2, 0.5, 1.0, 2.0]
	Measurement noise R	[0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2]
Particle Filter	Process std. σ_Q	[0.001, 0.005, 0.01, 0.05, 0.1, 0.2, 0.3]
	Measurement std. σ_R	[0.002, 0.005, 0.01, 0.02, 0.03]
	Number of particles N_p	[100, 250, 500, 750, 1000]

All predictors were initialized using the minimum observed value of each variable from the 2016 dataset and fixed with the optimal configurations identified through the grid search.

5.2 Results

This section presents the quantitative and qualitative results of the evaluation, comparing reconstruction quality and complex event detection accuracy across the oracle, MCAR, and structural-gap conditions.

5.2.1 Reconstruction Accuracy

Table 5.3: Comparison of reconstruction performance (MAE) per dataset and attribute (averaged across beaches) between Kalman and Particle filters under hybrid missingness.

Attribute	Kalman Filter			Particle Filter		
	hybrid_10	hybrid_20	hybrid_30	hybrid_10	hybrid_20	hybrid_30
Turbidity	0.037	0.090	0.141	X	X	X
Water Temperature	0.032	0.073	0.122	X	X	X
Wave Height	0.002	0.005	0.007	X	X	X
Wave Period	0.117	0.241	0.337	X	X	X

5.2.2 Event Continuity and Detection

Kalman Filter

5.2.3 Confidence Behavior

(Plots of confidence trajectories under varying Q and missingness rates.)

5.2.4 System-Level Observations

(Summarize emergent patterns across all beaches — e.g., correlation, stability, and temporal drift.)

Chapter 6

Conclusion

Every thesis also needs a concluding chapters

Bibliography

- [n. d.]. ZeroMQ: Distributed Messaging Library. <https://zeromq.org/>. Accessed: 2025-10-08.
2016. Reference Architecture Model Industrie 4.0 (RAMI 4.0).
2021. Automation systems and integration — Digital Twin framework for manufacturing — Part 1: Overview and general principles.
- Feyi Adesanya, Kanan Castro Silva, Valdemar V Graciano Neto, and Istvan David. 2025. Systems of Twinned Systems: A Systematic Literature Review. *arXiv preprint arXiv:2505.19916* (2025).
- V Akila, V Govindasamy, and S Sandosh. 2016. Complex event processing over uncertain events: Techniques, challenges, and future directions. In *2016 International Conference on Computation of Power, Energy Information and Commuincation (ICCPEIC)*. IEEE, 204–221.
- M Sanjeev Arulampalam, Simon Maskell, Neil Gordon, and Tim Clapp. 2002. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on signal processing* 50, 2 (2002), 174–188.

- Barbara Rita Barricelli, Elena Casiraghi, and Daniela Fogli. 2019. A survey on digital twin: Definitions, characteristics, applications, and design implications. *IEEE access* 7 (2019), 167653–167671.
- Heni Belgacem and Inès Chihi. 2025. Toward Reliable and Intelligent Sensor Systems: A Comprehensive Study of Fault Diagnosis and Mitigation. *IEEE Sensors Reviews* (2025).
- Stefan Boschert and Roland Rosen. 2016. Digital twin—the simulation aspect. *Mechatronic futures: Challenges and solutions for mechatronic systems and their designers* (2016), 59–74.
- Diego M Botín-Sanabria, Adriana-Simona Mihaita, Rodrigo E Peimbert-García, Mauricio A Ramírez-Moreno, Ricardo A Ramírez-Mendoza, and Jorge de J Lozoya-Santos. 2022. Digital twin technology challenges and applications: A comprehensive review. *Remote Sensing* 14, 6 (2022), 1335.
- Chicago Park District. 2025. Beach Water Quality - Automated Sensors. https://data.cityofchicago.org/Parks-Recreation/Beach-Weather-Stations-Automated-Sensors/k7hf-8y75/about_data. Data collected and maintained by the Chicago Park District; accessed September 18, 2025.
- T Chithrakumar, Sandeep Kumar Mathivanan, M Thangamani, Balamurugan Balusamy, Shilpa Gite, Nilkanth Deshpande, et al. 2024. Revolutionizing Agriculture through Cyber Physical Systems: The Role of Robotics in Smart Farming. In *2024 International Conference on Electrical Electronics and Computing Technologies (ICEECT)*, Vol. 1. IEEE, 1–6.

- Mashrur Chowdhury, Kakan Dey, and Amy Apon. 2024. *Data analytics for intelligent transportation systems*. Elsevier.
- Istvan David and Dominik Bork. 2024. Infonomics of Autonomous Digital Twins. In *International Conference on Advanced Information Systems Engineering (LNCS)*. Springer, 563–578.
- Marietheres Dietz and Günther Pernul. 2020. Digital Twin: Empowering Enterprises Towards a System-of-Systems Approach. *Business & Information Systems Engineering* 62, 2 (2020), 179–184. doi:10.1007/s12599-019-00624-0
- Lukas Esterle, Cláudio Gomes, Mirgita Frasheri, Henrik Ejersbo, Sven Tomforde, and Peter G. Larsen. 2021. Digital twins for collaboration and self-integration. In *2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*. 172–177. doi:10.1109/ACSOS-C52956.2021.00040
- Zhemei Fang, Jingjing Liao, and Xiaozhou Zhou. 2020. Improving system-of-systems agility through dynamic reconfiguration. In *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 4466–4472.
- Francisco Henrique Cerdeira Ferreira, Elisa Yumi Nakagawa, Antonia Bertolino, Francesca Lonetti, Vânia de Oliveira Neves, and Rodrigo Pereira dos Santos. 2024. A framework for the design of fault-tolerant systems-of-systems. *Journal of Systems and Software* 211 (2024), 112010.
- GE Vernova. 2024. Predix Platform Overview. https://www.gevernova.com/software/documentation/predix-platforms/PDFs/Predix_Overview.pdf. Accessed: October 10, 2025.

- David Gelernter. 1993. *Mirror worlds: Or the day software puts the universe in a shoebox... How it will happen and what it will mean*. Oxford University Press.
- Edward Glaessgen and David Stargel. 2012. The digital twin paradigm for future NASA and US Air Force vehicles. In *53rd AIAA/ASME/ASCE/AHS/ASC structures, structural dynamics and materials conference 20th AIAA/ASME/AHS adaptive structures conference 14th AIAA*. 1818.
- Michael W Grieves. 2005. Product lifecycle management: the new paradigm for enterprises. *International Journal of Product Development* 2, 1-2 (2005), 71–84.
- Karl A Hribernik, Lutz Rabe, Klaus-Dieter Thoben, and Jens Schumacher. 2006. The product avatar as a product-instance-centric information management concept. *International Journal of Product Lifecycle Management* 1, 4 (2006), 367–379.
- Yujiao Hu, Qingmin Jia, Yuan Yao, Yong Lee, Mengjie Lee, Chenyi Wang, Xiaomao Zhou, Renchao Xie, and F Richard Yu. 2024. Industrial internet of things intelligence empowering smart manufacturing: A literature review. *IEEE Internet of Things Journal* 11, 11 (2024), 19143–19167.
- INCOSE. 2023. *Systems Engineering Vision 2035: Engineering Solutions For A Better World*. Technical Report. [Online]. Available: <https://www.incose.org/docs/default-source/se-vision/incose-se-vision-2035.pdf>.
- Innovation, Science and Economic Development Canada and Canadian Forum for Digital Infrastructure Resilience. 2023. Improving the Reliability and Resilience of Canada’s Digital Infrastructure. <https://ised-isde.canada.ca/site/ised/en/>

- improving-reliability-and-resilience-canadas-digital-infrastructure.
Government of Canada, Ottawa.
- Wei Kang and Liang Xu. 2021. Some quantitative characteristics of error covariance for Kalman filters. *Tellus A: Dynamic Meteorology and Oceanography* 73, 1 (2021), 1–19.
- Barbara Kitchenham and Stuart Charters. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report EBSE-2007-01. Keele, UK, Keele University. 1–65 pages.
- Werner Kritzing, Matthias Karner, Georg Traar, Jan Henjes, and Wilfried Sihn. 2018. Digital Twin in manufacturing: A categorical literature review and classification. *Ifac-PapersOnline* 51, 11 (2018), 1016–1022.
- Jiewu Leng, Dewen Wang, Weiming Shen, Xinyu Li, Qiang Liu, and Xin Chen. 2021. Digital twins-based smart manufacturing system design in Industry 4.0: A review. *Journal of Manufacturing Systems* 60 (2021), 119–137. doi:10.1016/j.jmsy.2021.05.011
- Mengnan Liu, Shuiliang Fang, Huiyue Dong, and Cunzhi Xu. 2021. Review of digital twin about concepts, technologies, and industrial applications. *Journal of manufacturing systems* 58 (2021), 346–361.
- Mark W. Maier. 1998. Architecting principles for systems-of-systems. *Systems Engineering* 1, 4 (1998), 267–284.
- M Manikandan, P Venkatesh, T Illakya, M Krishnamoorthi, CR Senthilnathan, and K Maran. 2024. The significance of big data analytics in the global healthcare market.

- In *2024 International Conference on Communication, Computing and Internet of Things (IC3IoT)*. IEEE, 1–4.
- Hussein Marah, Lucas Lima, Hans Vangheluwe, and Moharram Challenger. 2025. An Agent-Oriented Twinning Architecture for Complex Event-Driven Anomaly Detection in Distributed CPS. In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 205–214.
- Claus Ballegaard Nielsen, Peter Gorm Larsen, John Fitzgerald, Jim Woodcock, and Jan Peleska. 2015. Systems of systems engineering: basic concepts, model-based techniques, and research directions. *ACM Computing Surveys (CSUR)* 48, 2 (2015), 1–41.
- OASIS Open. 2014. OASIS Standard: MQTT Version 3.1.1. <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>. OASIS Open Standard, Accessed: 2025-10-08.
- Luis Piardi, Paulo Leitao, Jonas Queiroz, and Joseane Pontes. 2024. Role of digital technologies to enhance the human integration in industrial cyber–physical systems. *Annual Reviews in Control* 57 (2024), 100934.
- Flávia Pires, Ana Cachada, José Barbosa, António Paulo Moreira, and Paulo Leitão. 2019. Digital twin in industry 4.0: Technologies, applications and challenges. In *2019 IEEE 17th international conference on industrial informatics (INDIN)*, Vol. 1. IEEE, 721–726.
- Mike Shafto, Michael Conroy, R Doyle, E Glaessgen, C Kemp, J LeMoigne, and

- L Wang. 2010. Modeling, Simulation, Information Technology and Processing Roadmap.
- Siemens. 2025. MindSphere Developer Documentation. <https://documentation.mindsphere.io/MindSphere/index.html>. Accessed: October 10, 2025.
- Zhanybek Suleiman, Sabit Shaikholla, Dinara Dikhanbayeva, Essam Shehab, and Ali Turkyilmaz. 2022. Industry 4.0: Clustering of concepts and characteristics. *Cogent Engineering* 9, 1 (2022), 2034264.
- Fei Tao, He Zhang, Ang Liu, and Andrew YC Nee. 2018. Digital twin in industry: State-of-the-art. *IEEE Transactions on industrial informatics* 15, 4 (2018), 2405–2415.
- Giuseppe Tricomi, Maurizio Giacobbe, Ilenia Ficili, Nicola Peditto, and Antonio Puliato. 2024. Smart city as cooperating smart areas: On the way of symbiotic cyber-physical systems environment. *Sensors* 24, 10 (2024), 3108.
- Gregory Vial. 2021. Understanding digital transformation: A review and a research agenda. *Managing digital transformation* (2021), 13–66.
- Claes Wohlin, Emilia Mendes, Katia Romero Felizardo, and Marcos Kalinowski. 2020. Guidelines for the search strategy to update systematic literature reviews in software engineering. *Information and Software Technology* 127 (2020), 106366.
- Meng Zhang, Fei Tao, Biqing Huang, Ang Liu, Lihui Wang, Nabil Anwer, and AYC Nee. 2024. Digital twin data: methods and key technologies: [version 2; peer review: 4 approved]. *Digital Twin* 1, 1 (2024), 2.